

Chapter 4: Program Control

C - How to Program, 9th edition

Umer Shahid

Department of Electrical Engineering,
UET Lahore

October 14, 2025

This Week's Topics

1. Introduction to Program Control
2. Iteration Essentials Review
3. Counter-Controlled Iteration
4. The for Loop
5. Examples Using for Statement
6. switch Statement
7. do-while Loop
8. break and continue
9. Logical Operators
10. Common Mistake: `=` vs `==`
11. Solved Problems

What We'll Learn This Week

- **New Loop Types:**

- `for` loop - Most commonly used loop
- `do...while` loop - Loop that runs at least once

- **Multiple Selection:**

- `switch` statement - Making choices from many options

- **Control Flow:**

- `break` - Exit loops early
- `continue` - Skip to next iteration

- **Logical Operators:**

- Combining conditions (`&&`, `—`, `!`)

What is Iteration? (Review)

Definition

Iteration (or looping) means repeating a group of instructions while some condition remains true.

Two Types We Know:

1. Counter-controlled:

- Know how many times to repeat
- Use a counter variable
- Example: Print 1 to 10

2. Sentinel-controlled:

- Don't know how many times
- Use a special "stop" value
- Example: Enter -1 to quit

Key Terms

- **Loop-continuation condition:** The test that keeps the loop running
- **Sentinel value:** Special value meaning "end of data"

Counter-Controlled Iteration - Requirements

Four Things You Always Need:

1. **Control Variable Name:** A variable to count (e.g., counter, i)
2. **Initial Value:** Where to start counting (e.g., 1)
3. **Increment/Decrement:** How much to change each time (e.g., +1, -1)
4. **Loop Condition:** When to stop (e.g., counter <= 5)

Simple Example with while

```
1 int counter = 1;           // 1. Name and initial value
2 while (counter <= 5) {     // 4. Loop condition
3     printf("%d ", counter);
4     ++counter;             // 3. Increment
5 }
6 // Output: 1 2 3 4 5
```

Important: Use Integer Counters!

Why Not Use Floating-Point Numbers?

- Floating-point numbers (like float, double) are **approximate**
- They may not be exactly what you expect
- This can cause loops to run wrong number of times

BAD - Don't Do This:

```
1 float x = 0.0;
2 while (x != 1.0) {
3     printf("%.1f ", x);
4     x += 0.1;
5 }
6 // May not stop at 1.0!
```

GOOD - Do This:

```
1 int i = 0;
2 while (i < 10) {
3     printf("%.1f ", i*0.1);
4     i++;
5 }
6 // Reliable!
```

Rule

Always use integer variables (int) for loop counters!

Introducing the for Loop

What is a for Loop?

The for loop is the most popular way to do counter-controlled iteration. It puts all the counter details in ONE line!

With while loop:

```
1 int counter = 1;
2 while (counter <= 5) {
3     printf("%d ", counter);
4     ++counter;
5 }
```

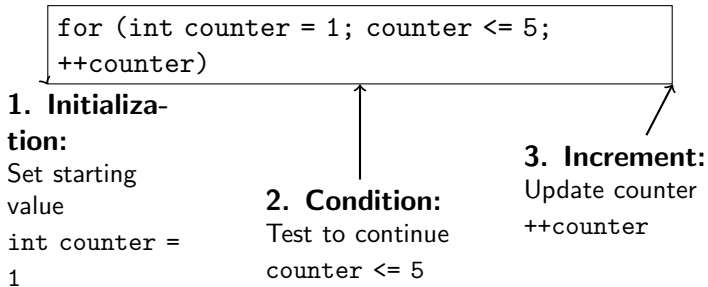
With for loop:

```
1 for (int counter = 1;
2     counter <= 5;
3     ++counter) {
4     printf("%d ", counter);
5 }
```

Both produce the same output

1 2 3 4 5

Understanding the for Loop Header



Remember

The three parts are separated by **semicolons (;)** - NOT commas!

How the for Loop Works - Step by Step

1. **Initialize:** Set counter = 1 (happens only once)
2. **Test:** Is counter <= 5?
 - If YES → Go to step 3
 - If NO → Exit the loop
3. **Execute:** Run the code inside { }
4. **Increment:** Do ++counter
5. **Repeat:** Go back to step 2

How the for Loop Works - Step by Step

Trace Example: `for (int i = 1; i <= 3; ++i)`

Step	Value of i	Action
Initialize	i = 1	Set initial value
Test	i = 1	1 i= 3? YES → Execute body
Increment	i = 2	Add 1 to i
Test	i = 2	2 i= 3? YES → Execute body
Increment	i = 3	Add 1 to i
Test	i = 3	3 i= 3? YES → Execute body
Increment	i = 4	Add 1 to i
Test	i = 4	4 i= 3? NO → EXIT

Complete for Loop Example

```
1 // fig04_02.c - Counter-controlled iteration with for
2 #include <stdio.h>
3
4 int main(void) {
5     // Print numbers 1 to 5
6     for (int counter = 1; counter <= 5; ++counter) {
7         printf("%d ", counter);
8     }
9
10    puts(""); // Print newline
11    return 0;
12 }
```

Output:

1 2 3 4 5

Scope of Control Variable

What is Scope?

Scope means "where a variable can be used"

Variable defined IN for:

```
1 for (int i = 1; i <= 5; ++i) {  
2     printf("%d", i);    // OK  
3 }  
4 printf("%d", i);    // ERROR!  
5 // i doesn't exist here
```

Variable defined BEFORE for:

```
1 int i = 1;  
2 for (; i <= 5; ++i) {  
3     printf("%d", i);    // OK  
4 }  
5 printf("%d", i);    // OK  
6 // i still exists (i = 6)
```

Best Practice

Define the counter in the for header unless you need it after the loop

Off-By-One Errors

Common Mistake: Off-By-One Error

This happens when your loop runs one time too many or too few!

Wrong - Runs 4 times:

```
1 // Want: 1 2 3 4 5
2 for (int i = 1; i < 5; ++i) {
3     printf("%d ", i);
4 }
5 // Output: 1 2 3 4
6 // Missing 5!
```

Correct - Runs 5 times:

```
1 // Want: 1 2 3 4 5
2 for (int i = 1; i <= 5; ++i) {
3     printf("%d ", i);
4 }
5 // Output: 1 2 3 4 5
6 // Perfect!
```

Tips to Avoid Off-By-One Errors

- Use `<=` when including the final value
- Use `<` when the final value should NOT be included
- Always test your loop with small numbers first!

General Format of for Statement

The Template

```
1 for (initialization; loopContinuationCondition; increment) {  
2     statement(s);  
3 }
```

- **initialization:** Create and set the counter (e.g., `int i = 0`)
- **loopContinuationCondition:** Test to keep going (e.g., `i < 10`)
- **increment:** Update the counter (e.g., `++i`, `i += 2`)
- The two semicolons `;` are **required!**

What if condition is initially false?

The loop body will **not execute at all** - the program skips to the next statement

Infinite Loops

What is an Infinite Loop?

A loop that **never stops** because its condition never becomes false

Example 1: Forgot increment

```
1 int i = 1;
2 while (i <= 5) {
3     printf("%d ", i);
4     // Forgot: ++i;
5 }
6 // Prints 1 forever!
```

Example 2: Wrong increment

```
1 for (int i = 1; i <= 5; --i) {
2     printf("%d ", i);
3 }
4 // i goes 1, 0, -1, -2...
5 // Never reaches end!
```

How to Stop an Infinite Loop on Your Computer

- Press **Ctrl + C** (hold Control and press C)
- Or close the terminal/command prompt window

The Increment Expression

All These Are Equivalent in a for loop:

```
1 counter = counter + 1
2 counter += 1
3 ++counter
4 counter++
```

All add 1 to counter

Decrementing (Counting Down)

```
1 // Count from 5 down to 1
2 for (int i = 5; i >= 1; --i) {
3     printf("%d ", i);
4 }
5 // Output: 5 4 3 2 1
```


Example 1: Different Increment Patterns

Count by 1:

```
1 for (int i = 1; i <= 100; ++i)
2 // 1, 2, 3, 4, ... 100
```

Count by 7:

```
1 for (int i = 7; i <= 77; i += 7)
2 // 7, 14, 21, 28, ... 77
```

Count by 3:

```
1 for (int j = 2; j <= 17; j += 3)
2 // 2, 5, 8, 11, 14, 17
```

Count down by 1:

```
1 for (int i = 100; i >= 1; --i)
2 // 100, 99, 98, ... 1
```

Count down by 2:

```
1 for (int i = 20; i >= 2; i -= 2)
2 // 20, 18, 16, ... 2
```

Count down by 11:

```
1 for (int j = 44; j >= 0; j -= 11)
2 // 44, 33, 22, 11, 0
```

Key Points

- Use += to count up by any amount
- Use -= to count down by any amount
- Use >= when counting down

Example 2: Sum of Even Integers (2 to 100)

Problem: Add all even numbers from 2 to 100

```
1 #include <stdio.h>
2
3 int main(void) {
4     int sum = 0;    // Initialize sum to 0
5
6     // Loop through even numbers: 2, 4, 6, ..., 100
7     for (int number = 2; number <= 100; number += 2) {
8         sum += number;    // Add current number to sum
9     }
10
11     printf("Sum is %d\n", sum);
12     return 0;
13 }
```

How it works:

$$\text{sum} = 0 + 2 + 4 + 6 + \dots + 98 + 100 = 2550$$

Example 3: Compound Interest

Problem: \$1000 invested at 5% interest for 10 years

```
1 #include <stdio.h>
2 #include <math.h> // For pow() function
3
4 int main(void) {
5     double principal = 1000.0; // Starting amount
6     double rate = 0.05;        // 5% interest rate
7
8     printf("%4s%21s\n", "Year", "Amount on deposit");
9
10    // Calculate for each year from 1 to 10
11    for (int year = 1; year <= 10; ++year) {
12        // Formula: amount = principal * (1 + rate)^year
13        double amount = principal * pow(1.0 + rate, year);
14
15        printf("%4d%21.2f\n", year, amount);
16    }
17    return 0;
18 }
```

Important

- Must include `#include <math.h>` for `pow()`
- `pow(x, y)` means x^y (x to the power of y)

Compound Interest - Output

Year	Amount on deposit
1	1050.00
2	1102.50
3	1157.63
4	1215.51
5	1276.28
6	1340.10
7	1407.10
8	1477.46
9	1551.33
10	1628.89

Compound Interest - Output

Understanding the Output

- Each year, money grows by 5%
- Year 1: $\$1000 \times 1.05 = \1050
- Year 2: $\$1000 \times 1.05^2 = \1102.50
- Year 10: $\$1000 \times 1.05^{10} = \1628.89

Formatting Output with printf

Format Specifier: %21.2f

- %f - floating-point number
- 21 - field width (total space used)
- .2 - precision (2 decimal places)
- Default: right-aligned (spaces on left)

Right-aligned (default):

```
1 printf("%10.2f\n", 123.5);
```

Output: 123.50

Left-aligned (use -):

```
1 printf("%-10.2f\n", 123.5);
```

Output: 123.50

Example

%21.2f reserves 21 character positions and shows 2 decimal places

What is Multiple Selection?

The Problem

Sometimes you need to check a variable against **many different values** and do different things for each value.

With if-else (tedious):

```
1 if (grade == 'A') {
2     ++aCount;
3 }
4 else if (grade == 'B') {
5     ++bCount;
6 }
7 else if (grade == 'C') {
8     ++cCount;
9 }
10 else if (grade == 'D') {
11     ++dCount;
12 }
13 else if (grade == 'F') {
14     ++fCount;
15 }
16 else {
17     printf("Invalid");
18 }
```

With switch (cleaner):

```
1 switch (grade) {
2     case 'A':
3         ++aCount;
4         break;
5     case 'B':
6         ++bCount;
7         break;
8     case 'C':
9         ++cCount;
10        break;
11    case 'D':
12        ++dCount;
13        break;
14    case 'F':
15        ++fCount;
16        break;
17    default:
18        printf("Invalid");
19        break;
20 }
```

Structure of switch Statement

```
1 switch (expression) {  
2     case value1:  
3         statement(s);  
4         break;  
5  
6     case value2:  
7         statement(s);  
8         break;  
9  
10    default:  
11        statement(s);  
12        break;  
13 }
```

- **expression:** Variable or expression to test
- **case:** Possible values to compare against
- **break:** Exit the switch (important!)
- **default:** What to do if no case matches (optional but recommended)

Complete switch Example - Letter Grades

```
1  #include <stdio.h>
2  int main(void) {
3      int aCount = 0, bCount = 0, cCount = 0;
4      int dCount = 0, fCount = 0;
5      puts("Enter the letter grades.");
6      puts("Enter EOF to end input.");
7      int grade;
8      while ((grade = getchar()) != EOF) {
9          switch (grade) {
10             case 'A': // Uppercase A
11                 case 'a': // Lowercase a
12                     ++aCount;
13                     break;
14             case 'B':
15                 case 'b':
16                     ++bCount;
17                     break;
18             // ... similar for C, D, F ...
19             case '\n': // Ignore newline
20             case '\t': // Ignore tab
21             case ' ': // Ignore space
22                 break;
23             default:
24                 printf("Incorrect grade entered.\n");
25                 break;
26         }
27     }
28     printf("A: %d\nB: %d\n", aCount, bCount);
29 }
```

Understanding getchar()

What is getchar()?

getchar() is a function that reads **one character** from the keyboard

```
1 int grade = getchar(); // Wait for user to type one character
```

- Returns the character as an int
- Returns EOF when user enters end-of-file signal

How to Enter EOF

- **Windows:** Press Ctrl + Z then Enter
- **macOS/Linux:** Press Ctrl + D

This tells the program "I'm done entering data"

Multiple Case Labels

Handling Similar Cases Together

You can list multiple case labels to do the same action for different values

```
1 switch (grade) {
2     case 'A':      // If grade is 'A'
3     case 'a':      // OR if grade is 'a'
4         ++aCount;   // Do this for both
5         break;
6     case '\n':     // If newline
7     case '\t':     // OR if tab
8     case ' ':      // OR if space
9         break;     // Exit switch
10    case 'B':
11        ++bCount;
12        break;
13 }
14 // If grade is 'A':
15 // Only aCount increases
```

The default Case

What is default?

The default case executes when **none** of the other cases match

```
1 switch (day) {
2     case 1:
3         printf("Monday\n");
4         break;
5     case 2:
6         printf("Tuesday\n");
7         break;
8     // ... cases 3-6 ...
9     case 7:
10        printf("Sunday\n");
11        break;
12    default:
13        printf("Invalid day! Enter 1-7\n");
14        break; // Optional but good practice
15 }
```

switch Restrictions

Important Limitations of switch

You can ONLY test:

- Integer values (int, short, long)
- Character values (char)
- **NOT** floating-point numbers (float, double)
- **NOT** strings

Valid:

```
1 switch (age) {  
2     case 18:  
3     case 19:  
4     case 20:  
5         // OK  
6 }  
7 switch (letter) {  
8     case 'A':  
9     case 'B':
```

Invalid:

```
1 switch (gpa) {  
2     case 3.5: // ERROR!  
3     case 4.0: // Can't use float  
4  
5 }  
6 switch (name) {  
7     case "Ali": // ERROR!  
8     case "Sara": // Can't use
```

switch vs if-else: When to Use Which?

Use switch when:

- Testing ONE variable
- Against many specific values
- Values are integers or characters
- Want cleaner, more readable code

Good for switch:

- Menu choices (1, 2, 3, 4)
- Letter grades (A, B, C, D, F)
- Day of week (1-7)

Use if-else when:

- Testing ranges (e.g., $x > 10$)
- Multiple conditions
- Floating-point comparisons
- String comparisons

Need if-else for:

- `if (age >= 18)`
- `if (temp > 100.5)`
- `if (x > 0 && y < 10)`

Introducing do-while Loop

Key Difference from while Loop

- **while loop:** Tests condition BEFORE running body
- **do-while loop:** Tests condition AFTER running body
- **Result:** do-while always runs **at least once!**

while loop:

```
1 int counter = 10;
2 while (counter < 5) {
3     printf("%d ", counter);
4     ++counter;
5 }
6 // Body never executes!
7 // (condition is false)
```

do-while loop:

```
1 int counter = 10;
2 do {
3     printf("%d ", counter);
4     ++counter;
5 } while (counter < 5);
6 // Prints "10" once!
7 // (runs before checking)
```

do-while Syntax

Format

```
1 do {  
2     statement(s);  
3 } while (condition); // Notice the semicolon!
```

Don't Forget!

- The semicolon ; after while (condition) is **required**
- This is different from regular while loops!

Complete Example

```
1 int counter = 1;  
2 do {  
3     printf("%d ", counter);  
4 } while (++counter <= 5);  
5
```


When to Use do-while

Perfect for Menus and Input Validation

Use do-while when you want the loop to run **at least once**

```
1 int choice;
2 do {
3     printf("\n--- MENU ---\n");
4     printf("1. Add\n");
5     printf("2. Subtract\n");
6     printf("3. Exit\n");
7     printf("Enter choice: ");
8     scanf("%d", &choice);
9
10    if (choice < 1 || choice > 3) {
11        printf("Invalid! Try again.\n");
12    }
13 } while (choice != 3); // Keep showing menu until user picks 3
```

Why do-while here?

Comparison: while vs do-while

Feature	while	do-while
Test condition	Before body	After body
Minimum executions	0 times	1 time
Semicolon after condition?	No	Yes (required)
Best for	Unknown iterations	Menu/validation

Example Comparison

Task: Get a positive number from user

With do-while: Ask at least once, repeat if invalid

With while: Would need to ask once before loop (duplicated code)

The break Statement

What does break do?

break immediately **exits** the current loop or switch statement

```
1 // Exit loop when x becomes 5
2 for (int x = 1; x <= 10; ++x) {
3     if (x == 5) {
4         break; // Exit the loop NOW!
5     }
6     printf("%d ", x);
7 }
8 printf("\nBroke out at x = 5");
9
10 // Output:
11 // 1 2 3 4
12 // Broke out at x = 5
```

- Loop stops immediately when break executes
- Control jumps to first statement **after** the loop
- Works in while, for, do-while, and switch

break Example - Finding First Occurrence

Task: Find if a number appears in a list

```
1 #include <stdio.h>
2
3 int main(void) {
4     int numbers[] = {3, 7, 2, 9, 15, 4, 8};
5     int target = 9;
6     int found = 0;    // Flag: 0 = not found, 1 = found
7
8     for (int i = 0; i < 7; ++i) {
9         if (numbers[i] == target) {
10             found = 1;
11             printf("Found %d at position %d\n", target, i);
12             break;    // No need to keep searching!
13         }
14     }
15
16     if (!found) {
17         printf("%d not found\n", target);
```

The continue Statement

What does continue do?

continue skips the **rest of current iteration** and starts the next one

```
1 // Skip printing 5
2 for (int x = 1; x <= 10; ++x) {
3     if (x == 5) {
4         continue; // Skip to next iteration!
5     }
6     printf("%d ", x);
7 }
8
9 // Output: 1 2 3 4 6 7 8 9 10
10 // (Notice 5 is missing!)
```

- Skips remaining code in loop body
- Goes directly to next iteration
- In for: increment happens, then condition is tested
- In while/do-while: condition is tested immediately

continue Example - Skip Even Numbers

Task: Print only odd numbers

```
1 #include <stdio.h>
2
3 int main(void) {
4     printf("Odd numbers from 1 to 10:\n");
5
6     for (int i = 1; i <= 10; ++i) {
7         if (i % 2 == 0) { // If i is even
8             continue;    // Skip the printf below
9         }
10        printf("%d ", i);
11    }
12    return 0;
13 }
14 // Output: Odd numbers from 1 to 10:
15 //        1 3 5 7 9
```

break vs continue

	break	continue
Action	Exit loop completely	Skip to next iteration
What happens next?	Jump to statement after loop	Jump to loop update/test
Use when	Found what you need	Want to skip certain values
Loop ends?	YES	NO (loop continues)

Visual Comparison

break: "Stop the loop entirely!"

continue: "Skip this one, but keep going!"

Introduction to Logical Operators

Why Do We Need Them?

Sometimes we need to test **multiple conditions** at once:

- Is age between 18 AND 65?
- Is grade 'A' OR 'B'?
- Is number NOT negative?

Operator	Name	Meaning
&&	Logical AND	Both conditions must be true
	Logical OR	At least one must be true
!	Logical NOT	Reverse the condition

Remember

These are **logical** operators (for true/false), not bitwise operators!

Logical AND (&&) Operator

How AND Works

Result is TRUE only if **BOTH** conditions are TRUE

```
1 int age = 25;
2 int hasLicense = 1; // 1 = true, 0 = false
3
4 // Check if person can rent a car
5 if (age >= 21 && hasLicense == 1) {
6     printf("You can rent a car\n");
7 }
```

Truth Table for &&

Condition 1	Condition 2	Result
FALSE	FALSE	FALSE
FALSE	TRUE	FALSE
TRUE	FALSE	FALSE
TRUE	TRUE	TRUE

More AND Examples

Example 1: Range Check

```
1 int score = 85;
2
3 // Check if score is between 60 and 100
4 if (score >= 60 && score <= 100) {
5     printf("Valid score\n");
6 }
```

More AND Examples

Example 2: Senior Female Count

```
1 int gender = 1; // 1 = female, 2 = male
2 int age = 68;
3
4 // Count senior females
5 if (gender == 1 && age >= 65) {
6     ++seniorFemales;
7     printf("Senior female counted\n");
8 }
```

Logical OR (——) Operator

How OR Works

Result is TRUE if **AT LEAST ONE** condition is TRUE

```
1 int semesterAverage = 85;
2 int finalExam = 92;
3
4 // Student gets 'A' if either average >= 90 OR final >= 90
5 if (semesterAverage >= 90 || finalExam >= 90) {
6     printf("Student grade is A\n");
7 }
```

Truth Table for ——

Condition 1	Condition 2	Result
FALSE	FALSE	FALSE
FALSE	TRUE	TRUE
TRUE	FALSE	TRUE
TRUE	TRUE	TRUE

More OR Examples

Example 1: Weekend Check

```
1 int day = 6; // 1=Mon, 2=Tue, ..., 6=Sat, 7=Sun
2
3 // Check if it's a weekend
4 if (day == 6 || day == 7) {
5     printf("It's the weekend!\n");
6 }
```

More OR Examples

Example 2: Emergency Exit

```
1 char response;  
2 printf("Continue? (y/n): ");  
3 scanf(" %c", &response);  
4  
5 // Exit if user enters 'n' or 'N'  
6 if (response == 'n' || response == 'N') {  
7     printf("Exiting program...\n");  
8     return 0;  
9 }
```

Logical NOT (!) Operator

How NOT Works

Reverses the condition: TRUE becomes FALSE, FALSE becomes TRUE

```
1 int isRaining = 0; // 0 = false, not raining
2
3 // If it's NOT raining
4 if (!isRaining) {
5     printf("Let's go outside!\n");
6 }
7
8 // This is the same as:
9 if (isRaining == 0) {
10     printf("Let's go outside!\n");
11 }
```

Truth Table for !

Condition	!Condition
FALSE	TRUE
TRUE	FALSE

NOT Operator Examples

Example 1: Input Validation

```
1 int validInput = 0;
2 int number;
3 scanf("%d", &number);
4 if (number >= 1 && number <= 10) {
5     validInput = 1;
6 }
7 // If NOT valid input
8 if (!validInput) {
9     printf("Error: Enter a number between 1 and 10\n");
10 }
```

Prefer Simpler Expressions

Instead of: `!(grade == sentinelValue)`

Write: `grade != sentinelValue`

Both work, but the second is clearer!

Short-Circuit Evaluation

What is Short-Circuit Evaluation?

C stops evaluating logical expressions as soon as the result is known

AND (&&) short-circuit:

```
1 if (x != 0 && y/x > 2) {  
2     // Safe!  
3 }
```

- If `x != 0` is FALSE
- Stop! (whole thing is FALSE)
- Never evaluate `y/x`
- Avoids division by zero!

OR (||) short-circuit:

```
1 if (age < 18 || hasPermission) {  
2     // Check permission  
3 }
```

- If `age < 18` is TRUE
- Stop! (whole thing is TRUE)
- Never check `hasPermission`
- Saves time!

Performance Tip

- AND: Put most likely to be FALSE first
- OR: Put most likely to be TRUE first

Combining Multiple Logical Operators

Use Parentheses for Clarity!

Even when not required by precedence, use `()` to make your intent clear:

- Good: `(a && b) || (c && d)`
- Confusing: `a && b || c && d`

Operator Precedence - Complete Table

Operators	Associativity	Type
++ -- (postfix)	Right to left	Postfix
! ++ -- (prefix) + (unary) - (unary)	Right to left	Unary
* / %	Left to right	Multiplicative
+ -	Left to right	Additive
< <= > >=	Left to right	Relational
== !=	Left to right	Equality
&&	Left to right	Logical AND
	Left to right	Logical OR
= += -= *= /= %=	Right to left	Assignment

What This Means

- ! has higher precedence than &&
- && has higher precedence than ||
- Relational operators (<, >) evaluated before logical operators

The Most Common C Programming Error!

Confusing = and ==

- == means "is equal to" (comparison)
- = means "assign value" (assignment)
- Swapping them creates LOGIC ERRORS (not syntax errors!)

WRONG (but compiles!):

```
1 int payCode = 3;
2
3 if (payCode = 4) {
4     printf("Bonus!\n");
5 }
6 // ALWAYS prints "Bonus!"
7 // payCode is now 4!
```

CORRECT:

```
1 int payCode = 3;
2
3 if (payCode == 4) {
4     printf("Bonus!\n");
5 }
6 // Only prints if payCode
7 // was already 4
```

Why the error?

Assignment `payCode = 4` has value 4, which is TRUE (nonzero)!

How to Avoid This Mistake

Defensive Programming Technique

Write constants on the LEFT side of comparisons

Normal way (risky):

```
1 if (x == 5) {  
2     // ...  
3 }  
4  
5 // If you accidentally type:  
6 if (x = 5) {  
7     // Compiles! ERROR!  
8 }
```

Safer way:

```
1 if (5 == x) {  
2     // ...  
3 }  
4  
5 // If you accidentally type:  
6 if (5 = x) {  
7     // COMPILER ERROR!  
8     // Can't assign to 5  
9 }
```

Remember

- **lvalue** = can be on left of = (variables)
- **rvalue** = can only be on right of = (constants, expressions)

Another Dangerous Mistake

Using == When You Mean =

What you want:

```
1 int x;  
2 x = 5; // Assign 5 to x  
3 printf("%d\n", x);  
4 // Output: 5
```

What you accidentally type:

```
1 int x = 3;  
2 x == 5; // Compare x to 5  
3 printf("%d\n", x);  
4 // Output: 3  
5 // x unchanged!
```

The Problem

- `x == 5` compares but does nothing with result
- No assignment happens
- Compiles without error
- `x` keeps its old value
- Many compilers will warn you about this

Problem 1: Sum Odd Integers (Exercise 4.3a)

Problem: Sum the odd integers between 1 and 99 using a for statement.

```
1 #include <stdio.h>
2
3 int main(void) {
4     int sum = 0;    // Initialize sum to 0
5
6     // Loop through odd numbers: 1, 3, 5, ..., 99
7     for (int count = 1; count <= 99; count += 2) {
8         sum += count;    // Add current odd number to sum
9     }
10
11     printf("Sum of odd integers from 1 to 99 is: %d\n", sum);
12     return 0;
13 }
```

Explanation

Start at 1, increment by 2 each time (1, 3, 5, ..., 99), add each to sum

Problem 2: Print Integers with Formatting (Exercise 4.3d)

Problem: Print integers from 1 to 20, five per line using tab separation.

```
1 #include <stdio.h>
2
3 int main(void) {
4     int x = 1;
5     while (x <= 20) {
6         printf("%d", x);        // Print the number
7         if (x % 5 == 0) {      // Every 5th number
8             printf("\n");      // Print newline
9         }
10        else {
11            printf("\t");       // Print tab
12        }
13        ++x; // Increment counter
14    }
15    return 0;
16 }
```